Gatekeeper

System Design Description

Abstract
This document describes the design and implementation of the Gatekeeper support system of Arrowhead 5.2.2 (`ah5-Gatekeeper-java-spring`).

Contents

1 Overview **3**

2 Implementation **4**
2.1 Implementation language and tools 4
2.2 Functional properties implementation 4
2.3 Non functional properties implementation 4
2.4 Deployment 5
2.5 Local modifications 5
2.6 Verification 5

3 Services **6**

4 References **6**

5 Revision History **8**
5.1 Amendments 8
5.2 Quality Assurance 8

1 Overview

This document describes the Gatekeeper system implementation, which realizes the Gatekeeper SysD [1]. The Gatekeeper connects the local cloud to its neighbour clouds through the inter-cloud MQTT relay: global service discovery of authorized foreign providers and inter-cloud negotiation of GateTunnel sessions. In Section 2, we describe implementation details of the system.

2 Implementation

This implementation realizes the Gatekeeper SysD in `ah5-Gatekeeper-java-spring`. The Java sources, dependencies, tests, image and deployment definition are owned by this folder. The previous implementation is preserved in `legacy/` and is excluded from the build.

2.1 Implementation language and tools

- Java 21 without a framework (JDK `HttpsServer`), Bouncy Castle, SQLite JDBC and RabbitMQ AMQP client; Eclipse Paho MQTT v3 relay client. Maven produces only `arrowhead-gatekeeper-5.2.2.jar`; the fixed `Main` entry point starts Gatekeeper. No environment variable selects another system.
- Storage: SQLite (SQLite with WAL on a data volume). Requests are handled on virtual threads with bounded parallelism.

2.2 Functional properties implementation

The following components implement the system:

Component	Responsibility
`Gatekeeper.java`	GSD, ICN, relay message handling, management.
`Messaging.java`	Relay and sysMonitor.
`PeerNames.java`	Peer cloud identities.

Every request passes `Service.handle`: request-count limit, `caller` (mutual TLS, certificate profile `sy` or `op`, identity token verified with Authentication and matched to the certificate), then the role's `route` with its caller lists.

2.2.1 Data handled and stored

Storage `SQLite`:

Tables	Content
`clouds`	Neighbour clouds.
`relays`	Relays.

The interfaces exchange JSON objects built at run time; their structure is specified in the IDD documents.

2.3 Non functional properties implementation

2.3.1 Security

`Tls` builds the TLS contexts from PEM files (`TLS_*`), with mandatory client certificates and CRL checking (`CRL_FILE`); `Service.caller` binds the identity token to the certificate; the Authentication client pins Authentication's certificate identity before sending credentials.

2.3.2 Power management

None; the system is a server process without power states. The `sysMonitor` `power` observation always reports line supply.

2.3.3 Internal monitoring

Logs go to stdout. The system publishes its `sysMonitor` observations every 30 s (see the sysMonitor IDD).

2.3.4 Configuration

Configuration is read from environment variables (listed in the SysD); secrets are read from files (`*_FILE` variables or mounted credential bundles).

2.4 Deployment

The local `Dockerfile` builds only this folder and runs `arrowhead-gatekeeper-standalone:5.2.2`. The local `compose.yaml` joins an operator-selected existing cloud network; it has no sibling build context, shared runtime image or dependency on another support process for startup. The HTTPS API uses port 8449; health uses internal port 8480. The process runs as UID 10001 with a read-only root filesystem and writable private `/data` and `/tmp` paths. The image preinstalls the platform SQLite native library in `/app/lib`; temporary storage remains non-executable. Certificates and credentials are mounted individually. This packaging is supplied for operator deployment; existing test-cloud deployments are separate deployment consumers and are not changed by it.

To select this system, copy `.env.example` to `.env`, provision its identity/password, system certificate/key, trusted CA and CRL, configure network and registry, then run `docker compose up -d --build` from this folder. The private project-scoped SQLite volume must never be shared with another system or concurrently mounted by a second instance. Stopping this Compose project preserves its volume and does not stop other systems.

ServiceRegistry is the bootstrap endpoint. Authentication and DynamicServiceOrchestration are looked up there; other consumed services are orchestrated through DSO and require the cloud's authorization rules. AMQPS monitoring remains a network dependency. Federation additionally requires the organization MQTT relay, configured neighbor identities, relay certificate/key and organization trust. The Compose default is AH-GT-1, with explicit legacy-mTLS compatibility; peers must agree and must never silently downgrade. Gatekeeper and GateTunnel cooperate through authorized network services, while retaining separate builds and deployments. SysMonitor reports degraded HTTP 503 when AMQP intake or MQTT output is disconnected. The other native systems return process liveness. Inspect registration, broker and orchestration status for operational readiness.

2.4.1 Ownership and build boundary

The helpers `Service`, `Tls`, `Json`, `Database` and `Messaging` are private source copies inside this project. The JAR contains no implementation of the other three former runtime systems. There is no shared runtime process, data directory or Maven parent. Maven dependencies remain external libraries. Security fixes to duplicated helpers must be propagated and verified in every affected project.

Build with Java 21 and `mvn -B clean verify`, or use the local Docker build. The packaged SQLite smoke check is `java -jar target/arrowhead-gatekeeper-5.2.2.jar storage-selftest`. The build requires no sibling support folder.

2.5 Local modifications

Notes on this implementation in the workspace:

- Native Java port; inter-cloud orchestration with DynamicServiceOrchestration (local extension).
- Services registered with version `5.2.2`.

2.6 Verification

The project owns its JUnit security tests and system-specific tests. Verification covers mandatory mutual TLS, certificate/token binding, pinned Authentication identity, DSO discovery, concurrent token renewal and startup of this actual system with isolated generated credentials and its own temporary database. Federation and monitoring projects retain their relevant peer, session or persistence tests. The distribution verification report records independent build, packaging, image and configuration results; it does not claim a live-cloud rollout.

3 Services

The implementation of the services is based on the SD and IDD documents referenced below.

Services produced	**SysD ref**	**SD ref**	**IDD ref**
`globalServiceDiscovery`	[1]	[2]	[3]
`interCloudNegotiation`	[1]	[4]	[5]
`gatekeeperManagement`	[1]	[6]	[7]
`sysMonitor`	[1]	[8]	[9]
Services consumed	**SysD ref**	**SD ref**	**IDD ref**
serviceDiscovery	[10]	[11]	–
identity	[12]	[13]	–
authorizationManagement	[14]	[15]	–
gateTunnelConnection	[16]	[17]	–
inter-cloud messages	Relay (MQTT)	–	–
sysMonitor (publisher)	AMQP broker (SysMonitor)	–	–

4 References

[1] J. Delsing, “Gatekeeper – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-Gatekeeper-java-spring/documentation/v5.2.2/Gatekeeper-v5.2.2-SysD.pdf.

[2] ——, “globalServiceDiscovery – Service Description (Gatekeeper),” Eclipse Arrowhead, SD 5.2.2, 2026, gatekeeper-v5.2.2-globalServiceDiscovery-SD.pdf.

[3] ——, “globalServiceDiscovery HTTP/TLS/JSON – Interface Design Description (Gatekeeper),” Eclipse Arrowhead, IDD 5.2.2, 2026, gatekeeper-v5.2.2-globalServiceDiscovery-HTTP-IDD.pdf.

[4] ——, “interCloudNegotiation – Service Description (Gatekeeper),” Eclipse Arrowhead, SD 5.2.2, 2026, gatekeeper-v5.2.2-interCloudNegotiation-SD.pdf.

[5] ——, “interCloudNegotiation HTTP/TLS/JSON – Interface Design Description (Gatekeeper),” Eclipse Arrowhead, IDD 5.2.2, 2026, gatekeeper-v5.2.2-interCloudNegotiation-HTTP-IDD.pdf.

[6] ——, “gatekeeperManagement – Service Description (Gatekeeper),” Eclipse Arrowhead, SD 5.2.2, 2026, gatekeeper-v5.2.2-gatekeeperManagement-SD.pdf.

[7] ——, “gatekeeperManagement HTTP/TLS/JSON – Interface Design Description (Gatekeeper),” Eclipse Arrowhead, IDD 5.2.2, 2026, gatekeeper-v5.2.2-gatekeeperManagement-HTTP-IDD.pdf.

[8] ——, “sysMonitor – Service Description (Gatekeeper),” Eclipse Arrowhead, SD 5.2.2, 2026, gatekeeper-v5.2.2-sysMonitor-SD.pdf.

[9] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (Gatekeeper),” Eclipse Arrowhead, IDD 5.2.2, 2026, gatekeeper-v5.2.2-sysMonitor-AMQP-IDD.pdf.

[10] ——, “ServiceRegistry – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-SysD.pdf.

[11] ——, “serviceDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-serviceDiscovery-SD.pdf.

[12] ——, “Authentication – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-SysD.pdf.

[13] ——, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-identity-SD.pdf.

[14] ——, “ConsumerAuthorization – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-SysD.pdf.

[15] ——, “authorizationManagement – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-authorizationManagement-SD.pdf.

[16] ——, “GateTunnel – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-GateTunnel-java-spring/documentation/v5.2.2/GateTunnel-v5.2.2-SysD.pdf.

[17] ——, “gateTunnelConnection – Service Description (GateTunnel),” Eclipse Arrowhead, SD 5.2.2, 2026, gateTunnel-v5.2.2-gateTunnelConnection-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-25	5.2.2	Independent Java build, private helpers and state, per-system deployment and configuration.	Codex (draft)
3	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	